Spend Pixels Where It Hurts: Hazard-Aware Rendering for Optical Long-Context Compression

Anonymous CVPR submission

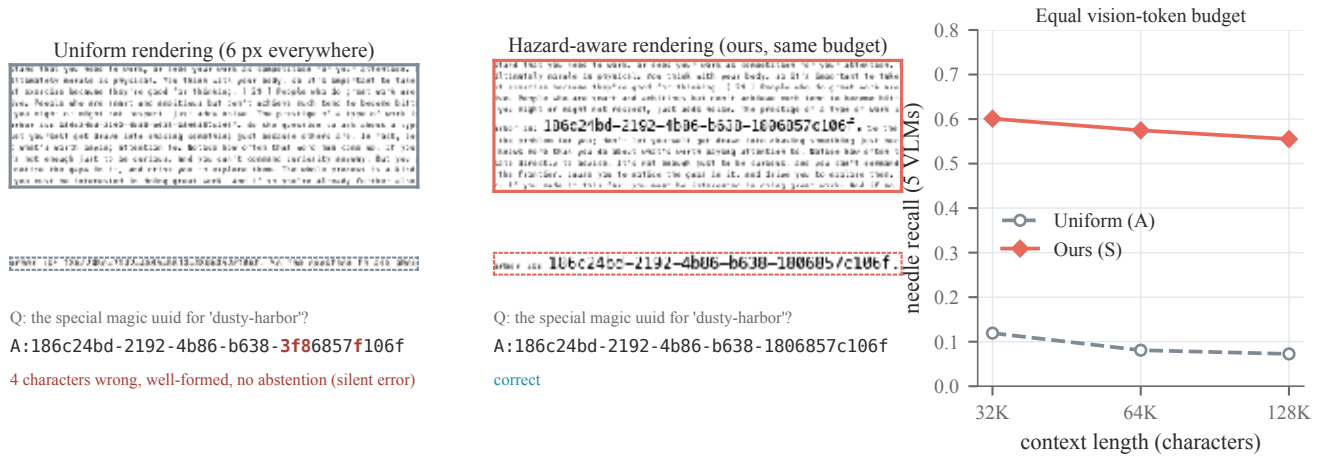


Figure 1. **Uniform rendering silently corrupts exactly the strings that must be copied.** A 32K-character essay hides one UUID. *Left:* rendered uniformly at 6 px, Claude Sonnet 5.5 returns a well-formed UUID with four wrong characters and no sign of doubt. *Middle:* our renderer enlarges the UUID within the same vision-token budget, and the answer is exact. *Right:* needle recall averaged over five VLMs and three RULER tasks. Uniform rendering degrades with context length while hazard-aware rendering does not.

Abstract

Rendering long text into images lets vision-language models (VLMs) read it with several times fewer tokens than the text itself. This optical compression spends resolution uniformly. Yet strings that must be copied exactly, such as identifiers and hashes, are misread into well-formed but wrong answers far more often than prose, and they fail more often as contexts grow. We ask where the pixels should go. The renderer, unlike the model, sees the text, so it can flag high-entropy strings and draw them larger while drawing prose smaller, within the vision-token budget of uniform rendering. This allocation is training-free, query-independent, and needs only black-box access to the VLM. Across five commercial and open-weight VLMs accessed through APIs, it raises RULER needle recall over uniform rendering by 0.55 and 0.43 at the two densest budgets (95% CIs [0.48, 0.62] and [0.27, 0.56]). At 32K–128K characters it raises recall by 0.48 while using up to 27% fewer billed tokens. Enlarging random words, or boxing the needles without enlarging them, does not help, so the gain comes from legibility of the targeted strings. The gain shrinks as the budget relaxes and is no larger than a font search at 10 px. On LongBench, allocation doubles the recall of naturally occurring identifiers that the detector flags (0.23 to 0.48) without changing QA accuracy relative to uniform rendering; strings the detector misses gain nothing. For flagged targets, verbatim anchors from prior work are as accurate on the allocated image as on a uniform one, and cheaper wherever allocation shrinks the image. All image arms remain well below plain text on QA: allocation makes optical compression safer for identifiers, not a substitute for text.

1. Introduction

Long contexts are expensive to feed to large models. Recent work cuts this cost by *rendering* text into images and letting a vision-language model (VLM) read the pixels [8, 34, 41]. A page of small glyphs costs a fixed number of vision tokens regardless of how many characters it holds, so dense rendering can compress a document several-fold. In our measurements, a 128K-character essay costs 27.1K billed tokens as text but only 4.7K as a uniformly rendered image

(mean of the four readers that report billed usage for both). The compression needs no training, works with any VLM that accepts images, and is controlled entirely by the party that renders the text.

Existing systems decide *how densely* to render, but rarely *where* the resolution goes. Optical compressors choose one font, size, and layout for the whole document, sometimes searched for a good accuracy–compression tradeoff [8]. Methods that do vary resolution within the input are trained [44, 46, 53] or read the model's attention for each query [48]. Visual-token reduction prunes or merges tokens after encoding [6, 47]. None of these is available to a party that only renders the text and calls a black-box API.

Uniform resolution hurts some strings far more than others. Identifiers, hashes, file paths, and long numbers carry no redundancy: a single misread glyph turns them into a different, equally plausible string. The surrounding prose survives the same blur. Figure 1 shows the typical outcome: a VLM returns a well-formed UUID with four wrong characters and no hedge. Prior work on failure-aware optical compression [1] showed that such silent corruption concentrates in high-entropy spans. It responded at the level of whole blocks: it flags these spans from the text, copies them verbatim next to the image as *anchors*, and routes risky blocks back to text. These remedies leave the image itself unchanged. Same problem found at long-context scale. At an equal budget, uniform rendering recovers only 8–22% of RULER needles (mean of five readers, Tab. 1), and its recall falls from 0.12 at 32K characters to 0.07 at 128K.

The renderer already holds the signal needed to fix the image. Before any pixel is drawn, the text says where the hazardous strings are, and the renderer decides how many pixels each of them gets. We therefore propose *hazard-aware rendering* (Fig. 2): flagged strings are drawn larger and prose smaller, subject to the image costing no more vision tokens than uniform rendering. The allocation (arm S in our experiments) is training-free and query-independent, so the same rendering can serve every question about a document. It composes with the anchors and text routing of prior work [1], which we treat as baselines and building blocks (arms K and X).

Our contributions are:

- **Budget-constrained glyph allocation** for rendered context, a rule that redistributes resolution inside the image using only the text and black-box access to the reader (Sec. 3).
- **Controlled evidence for where it helps.** Across five readers, allocation raises needle recall over uniform rendering by 0.43–0.55 at 16K characters and by 0.48 at 32K–128K. It beats font/size search, render-then-downsample, enlargement of random words, and boxing the needles without enlarging them (Tab. 2). The gain shrinks and matches font search at 10 px.
- **Natural strings and side effects.** On identifiers in LongBench documents that the detector flags, allocation doubles recall (0.23 to 0.48). On digit-bearing strings sampled independently of the detector, which it mostly does not flag, it neither helps nor hurts. It leaves LongBench QA unchanged relative to uniform rendering, while every image arm, ours included, trails plain text.
- **Composition with anchors.** Anchors are as accurate on the allocated image as on a uniform one (pooled X−K between −0.01 and +0.05 in every setting with flagged targets), and cost less wherever allocation shrinks the image. We also show that on standard needle tasks the anchors contain the answer, so recall there measures copying rather than reading, and that anchors can mislead readers when the target is not flagged.

2. Related Work

Optical context compression. Reading text from pixels has a long history in representation learning, from pixel-based language models [12, 34] to unified image–text encoders [37] and screenshot pretraining [21]. Recent systems use rendering explicitly to compress long contexts. DeepSeek-OCR decodes text from a small number of vision tokens [41], and its successor reorders visual tokens causally [42]. Glyph renders long inputs and searches for a good global rendering configuration [8]. Rendering a whole input as one image can roughly halve its token cost [26], and other work renders distant context or interleaves rendered text with images [39, 45]. Evaluations have started to show where this fails. Compressed reading loses retrieval ability [52], and accuracy collapses when linguistic priors are removed [13, 27]. The text–pixel gap depends strongly on font and resolution [36], and optical compression can underperform simple alternatives such as mean pooling [20]. Several methods vary resolution within the input. FocusVTC combines a low-resolution global view with adaptively enhanced regions [53], SEER learns query-dependent selective compression [46], and LensVLM post-trains a reader to expand relevant compressed pages on demand [44]. In document understanding, DocOwl2 and TextMonkey compress the visual tokens of high-resolution pages with learned modules [17, 29]. Closest to our rendering change, AGAR re-renders a page with the word spans that receive the most middle-to-late-layer attention enlarged [48]; it is training-free but needs the model's attention and a first pass for each query. Our allocation needs neither. It is decided from the text before any model call, works with black-box APIs, and does not depend on the question, so one rendering serves every question about a document (we do not measure provider-side caching). Failure-aware optical compression [1] is the closest prior work. It detects high-entropy spans from the text, keeps them as exact text anchors, routes risky blocks to text, and retries on

mismatch, but leaves the rendered image unchanged. We change the image and reuse its anchors and routing as baselines and building blocks.

Training-free visual token reduction. A complementary line of work reduces the tokens of an already encoded image. FastV drops low-attention tokens after an early layer [6], SparseVLM uses text guidance [50], VisionZip and PruMerge keep dominant or merged tokens [35, 47], and ToMe merges similar tokens [5]. Recent methods balance importance and diversity or target the output objective [11, 30, 49]. RTPrune adapts pruning to DeepSeek-OCR by exploiting text density [38], while FCoT-VL shows that text-rich images need dedicated compression [23]. These methods act on visual features inside the model, where it is no longer explicit that a group of glyphs forms a hash, and most need access to the model. Our method acts on the text, before encoding; we do not test whether pruning keeps or discards the enlarged glyphs.

Text-side and memory compression. Prompt compressors drop tokens judged uninformative by a small language model [18, 19, 24, 33]. Learned compressors map context to a few soft tokens [9, 14, 32], and KV-cache methods evict entries during inference [10, 25, 43, 51]. Selective Context, like our detector, uses an information-theoretic signal, but uses it to *remove* low-information text. Verbatim strings are high-information and must not be removed; we keep every character and only change how large it is drawn. For repeated queries over a fixed document, reusing cached prompt segments [15] is a competing way to cut cost that we do not evaluate.

Evaluation and input resolution. We measure verbatim fidelity with RULER needle tasks [16], whose UUID and number values are exactly the strings at risk, and collateral damage with LongBench QA [4]. Text-rich and long-document benchmarks [28, 31] document the same weakness in natural images and long PDF documents. Modern VLMs accept dynamic resolutions and allocate vision tokens roughly in proportion to pixel area, directly or through tiling [2, 3, 7, 22, 40]. The rendering budget is therefore a controllable cost. We report fidelity on synthetic needles and on naturally occurring identifiers separately from collateral damage, and we measure cost in provider-billed tokens rather than in pixels.

3. Method

Overview. Given a long document, an optical compressor renders it into image pages that a VLM reads in place of the text. Our goal is to make the strings that must be copied exactly survive this rendering, without spending more vision tokens and without access to the model. Figure 2 shows the pipeline. DETECT reads the text and marks verbatim-hazardous strings. ALLOCATE, our contribution, renders those strings large and the remaining prose small, so that the image costs no more than uniform rendering. The other two steps follow prior work on failure-aware optical compression [1]. ANCHOR appends verbatim text copies of the flagged strings, paid for here by the vision tokens that allocation saves. ROUTE falls back to plain text when the anchors would be too expensive.

3.1. Problem formulation

A document $x = (w_1, \ldots, w_n)$ is a sequence of whitespace-separated words. A renderer R maps x and a size assignment $\sigma : \{1, \ldots, n\} \to \mathbb{N}$ (glyph height in pixels for every word) to an image $I = R(x; \sigma)$ of fixed width, which is split into pages for the VLM. To compare renderings, we need their cost in the unit providers bill. Following the patch-based accounting of dynamic-resolution VLMs [40], we define the vision-token cost of an image with pages of width w_p and height h_p as

$$v(I) \;=\; \sum_{p} \left\lceil \frac{w_p}{28} \right\rceil \left\lceil \frac{h_p}{28} \right\rceil , \qquad (1)$$

where each vision token covers a 28×28-pixel area (14-pixel patches merged 2×2 in Qwen2-VL). Equation (1) is used only to enforce budgets; all costs we report are the prompt tokens billed by each provider. The *budget* is the cost of uniform rendering at a base size s_0, $B = v\big(R(x; \sigma \equiv s_0)\big)$. Every method we compare uses the same number of vision tokens as the uniform baseline, up to page rounding. Let $\mathcal{H}(x) \subseteq \{1, \ldots, n\}$ be the indices of hazardous words, those whose value is needed verbatim downstream. We seek a size assignment that makes every w_i, $i \in \mathcal{H}(x)$, as legible as possible subject to $v(I) \le B$, without deleting or reordering any character.

3.2. DETECT: text-side hazard detection

The detector approximates $\mathcal{H}(x)$ without labels, following the regex-plus-windowed-entropy design of [1]. Its premise is that a string is hazardous when it is *high-entropy*: a misread character yields another valid-looking string, and the language model cannot repair it from context. After stripping surrounding punctuation, we flag a word if it matches one of five patterns: a hexadecimal run of at least 8 characters, a run of at least 5 digits, a path containing a digit or a file extension, an IP address, or a digest prefix. We also flag words that mix letters and digits and either have at least 20 identifier characters or contain a 16-character window with Shannon entropy of at least 3.5 bits. Two choices encode what *not* to protect. Four-digit numbers are excluded because in natural text they are mostly years, which are low-entropy and cheap to read. The length and entropy rules

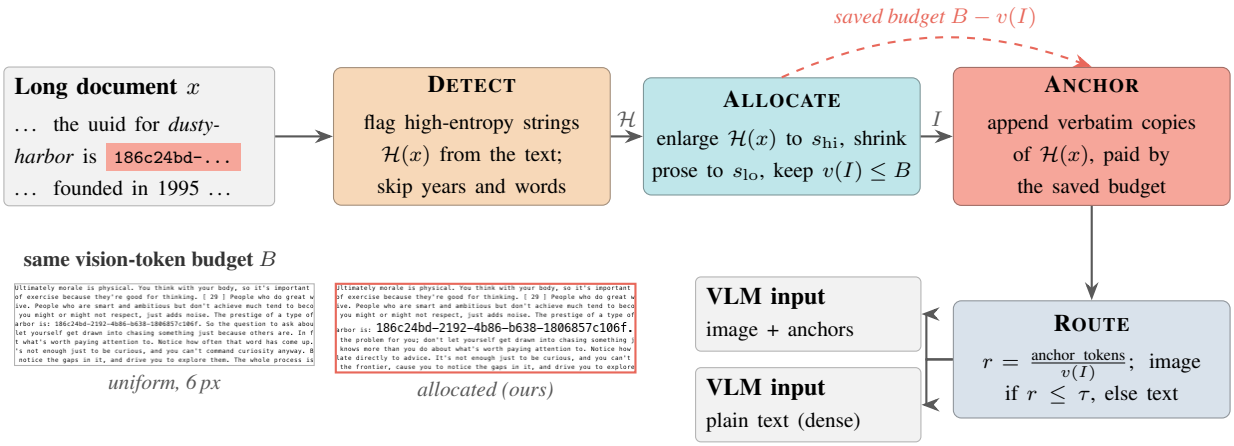


Figure 2. **Hazard-aware rendering.** DETECT marks verbatim-risky strings in the text before rendering. ALLOCATE, our contribution, enlarges only those strings and shrinks prose so that the image stays within the vision-token budget B of uniform rendering (up to page rounding). ANCHOR and ROUTE follow prior work [1]: they spend the saved budget on verbatim copies of the flagged strings and send documents whose anchors would be too expensive to plain text. The thumbnails show the same 32K-character document rendered uniformly and by our allocation. Every step is training-free and needs no access to the model.

require mixed letter–digit tokens so that long English words are not flagged. These exclusions are our changes. A preliminary detector without them flagged 85–112 words per LongBench document on average, most of them years, and shrank prose in documents that did not need it (Sec. I).

When the detector flags nothing, the rendering is identical to the uniform baseline, so a document whose hazards all go undetected loses only the potential gain (Sec. 5.7). A missed hazard in a document where *other* strings are flagged, however, shrinks together with the prose.

3.3. ALLOCATE: budget-constrained glyph allocation

Given $\mathcal{H}(x)$, we restrict size assignments to two levels: prose words at s_{lo} and flagged words at s_{hi}. Two levels keep the layout simple; a locally varying alternative that shrinks only prose near flagged strings brought no further gain (Sec. I). Prose is wrapped at character boundaries, as in the uniform baseline, so it consumes no extra space. Each flagged word is kept on one line and is never split. We choose the sizes by

$$(s^*_{\text{hi}}, s^*_{\text{lo}}) = \operatorname*{lexmax}_{\substack{s_{\text{hi}} \in [s_0,\, 12] \\ s_{\text{lo}} \in [5,\, \min(s_{\text{hi}}, s_0)]}} (s_{\text{hi}}, s_{\text{lo}}) \quad \text{s.t. } v\big(R(x; \sigma_{s_{\text{lo}}, s_{\text{hi}}})\big) \le B, \tag{2}$$

where $\sigma_{s_{\text{lo}}, s_{\text{hi}}}$ assigns s_{hi} to words in $\mathcal{H}(x)$ and s_{lo} to all others, and lexmax first maximizes s_{hi} and then s_{lo}. The order encodes a design choice: the marginal pixel is worth more on a hash than on prose. Verbatim accuracy is limited by the least legible character of a string, whereas prose comprehension degrades gracefully. The lower bound of 5 px keeps prose within the range VLMs still read, and $s_{\text{lo}} \le s_0$ ensures prose is never enlarged. If no pair with $s_{\text{hi}} > s_{\text{lo}}$ satisfies the constraint, prose-only lines lose their single pixel of extra line spacing and the search is repeated (in the implementation, s_{hi} is searched over $[6, 12]$). If $\mathcal{H}(x) = \emptyset$, the result is identical to uniform rendering. At $s_0 = 5$ px, prose is already at its minimum size, so this removed line spacing is what makes room for the enlarged strings. At $s_0 = 6$ px, prose shrinks to 5 px when needed and otherwise stays at 6 px. Budget computation plus allocation takes about 1 s for 32K and 4 s for 128K characters on one CPU core (unoptimized Python). Figure 1 (middle) shows a typical outcome: the UUID is drawn at 12 px while prose stays at 6 px.

3.4. ANCHOR and ROUTE

When prose can shrink ($s_0 = 6$ px), the allocated image usually ends *below* the budget, because the search stops at the largest feasible s_{hi} rather than filling every remaining row. In our long-context runs, it used 13–27% fewer billed tokens than uniform rendering. Following [1], we spend these savings on *anchors*: after the question, the flagged strings are appended verbatim as text, under a line stating that they are copied exactly from the image. Anchors carry no key, but a document with only one or a few flagged strings yields a short list that may consist of the answer alone. A reader can then answer without reading the image (Sec. 5.5). Anchor cost grows linearly with the number of flagged strings, so before any model call we compute the ratio

$$r(x) = \frac{\sum_{i \in \mathcal{H}(x)} c(w_i)}{v(I)}, \qquad c(w) = \tfrac{|w|}{2} + 3, \tag{3}$$

where $c(w)$ estimates the text tokens of one anchor and I is the allocated image, and send the document as plain text when $r(x) > \tau$. This replaces the density test of [1] with one that accounts for the allocated image. We use $\tau = 0.3$, the median of per-reader two-fold cross-validation on a synthetic density sweep (Sec. J).

4. Experimental Setup

Readers. We treat every VLM as a black box and query it with one stateless request per item at temperature 0 (0.01 for MiniMax). A request whose output cannot be parsed is resent up to three times. The six readers span vendors, scales, and open weights: Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, GPT-6 Luna (minimal reasoning effort), Qwen3-VL-235B-A22B [2], Llama 4 Maverick, and MiniMax-M3. All means and pooled tests use the five readers other than Claude, which was not run on every arm and is reported per reader only. Costs are the prompt tokens billed by each provider, which include the image, the question, and any appended copy. MiniMax does not report usage and is excluded from cost averages only. Empty responses and transport errors (fewer than 100 of roughly 35,000 calls in all experiments reported here, about 0.25%) are dropped. Any other unparseable output, including refusals and degenerate repetition, counts as wrong.

Tasks. We measure verbatim fidelity with RULER needle-in-a-haystack tasks [16], re-implemented with the official templates and Paul Graham essays as the haystack. The tasks are *single* (one UUID value), *multi-key* (four keys, one queried, numeric values), and *multi-value* (four numeric values for one key). Contexts have 16K characters (Sec. 5.1), 32K–128K characters split into up to three pages (Sec. 5.2), or 32K characters at milder budgets (Sec. 5.3). We measure collateral damage on ordinary comprehension with LongBench [4] MultiFieldQA-en and HotpotQA, from the length-balanced LongBench-E split. We replace non-ASCII characters by "?" and use contexts of at most 40K characters so that one image holds a document. These samples were drawn after the detector was finalized. We use $N = 25$ items per task and reader for RULER at 16K and 32K characters, $N = 20$ for the 32K–128K length study, and $N = 50$ for LongBench.

Arms. All image arms share the renderer of Sec. 3, a 1008-pixel page width, and the budget B of uniform rendering at $s_0 \in \{5, 6\}$ px (≈32 and 21 characters per vision token on our long documents). **A** is uniform rendering. **U** searches font family (monospace, Arial, Arial Narrow) and size for the largest uniform text that fits B, a training-free stand-in for global configuration search [8]. **Z** renders at 10 px and downsamples the image to B, the resolution-compression view of optical encoders [41]. **R** applies our allocation to randomly chosen ordinary words, matched to within 10% of the total length of the flagged strings. **B** uses S's layout without enlarging anything and boxes each flagged string, a salience-only control. **S** is our allocation (Sec. 3.3). **K** and **X** append anchors [1] to A and to S, and **T** gives the plain text. Prompts follow the task templates, with the context replaced by the image; RULER answers are requested as JSON.

Metrics and statistics. RULER is scored by recall of the gold values (case-insensitive containment, all values), LongBench by token-level F1, as in the original benchmarks. Because recall does not penalize extra answers, we also report a precision-aware F1 over the returned values. For each reader on each task at each setting (a *cell*) we compute the mean paired difference over items with a 95% bootstrap confidence interval (10,000 resamples). Pooled effects average over cells, with 95% CIs from a bootstrap that resamples both readers and items. We count significant cells after Holm correction within each comparison (Tab. 2).

5. Results

5.1. Equal-budget comparison

Table 1 compares all image arms at the same vision-token budget on 16K-character RULER documents. Averaged over five readers, uniform rendering recovers 8% of needles at the denser budget and 22% at the lighter one. Claude Sonnet 5.5 (0.57) and Gemini (0.88) are the only readers above 0.11 at 6 px. Allocation (S) raises the mean to 0.64 and 0.65, a pooled gain of 0.55 and 0.43 whose confidence intervals exclude zero by a wide margin (Tab. 2). After Holm correction, the gain is significant in 14 and 11 of 15 cells and never significantly negative. The two stronger uniform baselines barely help: font/size search (U) reaches 0.12 and 0.21, and render-then-downsample (Z) 0.16 and 0.19. Both plausibly fall short because they spread legibility evenly over all characters. At 6 px, S is also 8% cheaper than uniform rendering (842 vs. 920 billed tokens), because prose shrinks more than the enlarged strings grow. The weaker readers gain most (at 6 px Llama 4 Maverick from 0.00 to 0.59, GPT from 0.11 to 0.65), while Gemini, already at 0.88, gains 0.12. Under the precision-aware F1, which penalizes returning several candidate values, the gains are smaller (0.46 and 0.34 pooled) but remain significant in at least 9 of 15 cells.

Targeting, enlargement, or salience? Two controls separate the ingredients of S. Arm R runs the same allocation on randomly chosen ordinary words, matched in total length, so its sizes and cost are close to those of S while the needles shrink with the prose. R recovers 0.10 of the needles at both budgets, and S exceeds it by 0.54 and 0.56.

Table 1. **Equal-budget comparison on RULER needle retrieval (16K characters).** Recall per reader averaged over three NIAH tasks (N=25 each), at two budgets (uniform 5 px ≈ 32 and 6 px ≈ 21 characters per vision token). A, U, Z, R, and S are images with the same budget: uniform, best uniform font/size, render-then-downsample, *random* ordinary words enlarged by our allocation (matched in total length), and flagged strings enlarged (ours). K and X append anchors [1] to A and S. **Bold**: best within each group. Means, F1 (precision-aware: extra returned values count against the reader), and billed tokens are over the five readers other than Claude‡, which was not run on R and X. Pooled over tasks and readers, S−A is +0.55 [0.48, 0.62] at 5 px and +0.43 [0.27, 0.56] at 6 px (95% CI, readers and items resampled; Tab. 2). On these tasks the anchors contain the answer (Sec. 5.5), so K and X measure copying rather than reading.

	5 px budget (≈32 chars/vtok)							6 px budget (≈21 chars/vtok)						
	equal-budget image					+ anchors		equal-budget image					+ anchors	
Reader	A	U	Z	R	S (ours)	K	X	A	U	Z	R	S (ours)	K	X
Claude Sonnet 5.5‡	0.14	0.22	0.19	–	**0.79**	**0.95**	–	0.57	0.64	0.46	–	**0.92**	**1.00**	–
Gemini 3.5 Flash-Lite	0.41	0.57	0.79	0.48	**1.00**	**1.00**	**1.00**	0.88	0.78	0.84	0.46	**1.00**	1.00	**1.00**
GPT-6 Luna	0.02	0.03	0.02	0.02	**0.50**	0.91	**0.94**	0.11	0.16	0.08	0.02	**0.65**	0.82	**0.98**
Qwen3-VL-235B	0.00	0.00	0.00	0.00	**0.55**	**1.00**	**1.00**	0.07	0.08	0.03	0.00	**0.60**	**1.00**	**1.00**
Llama 4 Maverick	0.00	0.00	0.00	0.00	**0.58**	**0.97**	**0.97**	0.00	0.01	0.00	0.00	**0.59**	0.90	**1.00**
MiniMax-M3	0.00	0.00	0.00	0.00	**0.57**	**1.00**	0.98	0.02	0.01	0.00	0.00	**0.42**	0.97	**1.00**
Mean recall	0.08	0.12	0.16	0.10	**0.64**	0.97	**0.98**	0.22	0.21	0.19	0.10	**0.65**	0.94	**1.00**
Mean F1	0.09	0.12	0.16	0.10	**0.55**	0.80	**0.80**	0.23	0.21	0.19	0.10	**0.56**	0.79	**0.83**
Billed tokens	815	811	804	814	818	860	857	920	902	894	844	842	968	881



Figure 3. **Context length.** RULER at 32K–128K characters (6 px budget, up to three image pages, N=20 per task). (a) Needle recall; thin lines are individual readers (including Claude), thick lines the mean over five readers. (b) Billed prompt tokens averaged over the four of these readers that report usage; log scale.

Arm B keeps every glyph at the uniform size but draws a box around each flagged string, which makes the needle easy to *find* but not easier to *read*. B is 0.53 below S and slightly worse than uniform rendering (−0.05, CI upper bound −0.001; Tab. 2), perhaps because the box line touches the small glyphs. The gain therefore comes from making the targeted strings legible, not from layout changes or from pointing at the needle.

5.2. Scaling with context length

Optical compression is most attractive for long documents, so Fig. 3 extends the comparison to 128K characters. Under uniform rendering, mean recall declines from 0.12 at 32K to 0.08 at 64K and 0.07 at 128K, consistent with reports that compressed reading loses retrieval ability as context grows [52]. Allocation stays between 0.60 and 0.56, a pooled gain of 0.48 [0.43, 0.53] with 35 of 45 cells significant after Holm correction. It also uses 13%, 23%, and 27% fewer billed tokens than uniform rendering at the three lengths. On the four readers with billed costs for both, it uses 5.4×, 6.8×, and 8.0× fewer than plain text, which every reader answers with recall of at least 0.99. Individual readers differ widely (0.47 to 0.97 at 128K for S; Tab. 5).

5.3. Milder budgets

Our main budgets put most readers near the floor of uniform rendering, where gains are easiest to obtain. We therefore repeated the single and multi-key tasks at 32K characters with base sizes of 8 and 10 px, which cost 1.5 and 2.3 times as many vision tokens as 6 px. Uniform rendering improves with size (multi-key: 0.26 at 8 px, 0.60 at 10 px), and the gain of S shrinks accordingly, to 0.38 at 8 px and 0.20 at 10 px (Tab. 2). The font/size search catches up: S beats U by 0.26 at 8 px but only by 0.04 at 10 px, which is not significant. Single UUIDs remain hard even here (A 0.20, S 0.44 at 10 px). Allocation is most useful where the budget is tight, which is also where optical compression saves the most.

5.4. Naturally occurring strings

RULER needles are synthetic, and our detector was designed with such strings in mind. We built two sets of 60

Table 2. **Pooled effects** over the five non-Claude readers (recall unless noted). CI: 95% bootstrap resampling readers and items. Sig.: cells (reader × task) significantly positive after Holm correction within each row; neg.: significantly negative. All X−K comparisons we ran are listed.

Comparison	Δ	95% CI	sig.	neg.
Targeting vs. uniform and alternatives (RULER)				
S−A, 16K, 5 px	+0.55	[+0.48, +0.62]	14/15	0
S−A, 16K, 6 px	+0.43	[+0.27, +0.56]	11/15	0
S−A, 32K–128K, 6 px	+0.48	[+0.43, +0.53]	35/45	0
S−U (font/size search), 6 px	+0.44	[+0.31, +0.55]	11/15	0
S−R (random words), 6 px	+0.56	[+0.47, +0.64]	12/15	0
B−A (box, no enlargement), 6 px	-0.05	[-0.13, 0.00]	0/15	1
S−B, 6 px	+0.53	[+0.41, +0.64]	14/15	0
Milder budgets (32K)				
S−A, 8 px	+0.38	[+0.28, +0.46]	8/10	0
S−U, 8 px	+0.26	[+0.14, +0.40]	3/10	0
S−A, 10 px	+0.20	[+0.11, +0.30]	4/10	0
S−U, 10 px	+0.04	[-0.04, +0.12]	1/10	1
Natural strings (LongBench documents)				
S−A, flagged targets	+0.25	[+0.12, +0.38]	4/5	0
S−A, detector-independent	-0.01	[-0.11, +0.08]	0/5	0
Ordinary comprehension (LongBench F1)				
S−A	+0.02	[-0.02, +0.06]	0/10	0
X−A	+0.02	[-0.03, +0.07]	0/10	0
X−T (plain text)	-0.21	[-0.29, -0.13]	0/10	9
Anchors: X−K (F1 on RULER, recall on natural strings)				
RULER 16K, 5 px	0.00	[-0.02, +0.03]	0/15	0
RULER 16K, 6 px	+0.04	[0.00, +0.08]	1/15	0
RULER 32K, 8 px	0.00	[-0.04, +0.04]	0/10	0
RULER 32K, 10 px	0.00	[-0.05, +0.06]	0/10	0
density $k \leq 16$, 32K	-0.01	[-0.07, +0.04]	1/15	0
multi-key 64K–128K	-0.01	[-0.05, +0.03]	0/10	0
natural, flagged, 5 px	+0.04	[-0.03, +0.11]	0/5	0
natural, flagged, 6 px	+0.02	[-0.06, +0.10]	0/5	0
natural, flagged, 7 px	+0.05	[-0.06, +0.16]	1/5	0
natural, detector-indep.	-0.08	[-0.17, +0.01]	0/5	1

Table 3. **Naturally occurring strings.** Exact-copy recall on LongBench documents (≤40K characters, 6 px budget, N=60 each). *Flagged*: targets the detector flags (report and grant numbers, ISBNs, URLs, . . .). *Detector-independent*: any digit-bearing token, sampled without the detector (1/60 flagged; mostly short numbers and versions). */†: paired 95% CI vs. A above/below zero. Billed tokens: four readers that report usage.

	Flagged targets					Detector-independent targets				
Reader	A	S	K	X	T	A	S	K	X	T
Gemini 3.5 Flash-Lite	0.72	0.80	0.87*	0.90*	0.92	0.85	0.87	0.82	0.73†	1.00
GPT-6 Luna	0.25	0.55*	0.85*	0.82*	1.00	0.57	0.52	0.62	0.40†	0.98
Qwen3-VL-235B	0.12	0.48*	0.77*	0.78*	0.98	0.33	0.43	0.32	0.28	0.98
Llama 4 Maverick	0.05	0.20*	0.55*	0.62*	0.97	0.25	0.25	0.23	0.23	0.88
MiniMax-M3	0.03	0.39*	0.65*	0.67*	0.98	0.27	0.15	0.22	0.17	0.93
Mean	0.23	0.48	0.74	0.76	0.97	0.45	0.44	0.44	0.36	0.96
Billed tokens	1181	1035	1267	1128	5324	1111	1067	1225	1173	4942

questions from LongBench documents of up to 40K characters (ten each from six tasks). Each asks for the exact string that follows a unique six-word phrase (Sec. K). In the first set, targets are strings the detector flags, such as report and grant numbers, ISBNs, citation keys, and URLs. Here allocation doubles mean recall (0.23 to 0.48, significant for four of five readers) at 12% fewer billed tokens than uniform rendering (Tab. 3). In the second set, targets are any digit-bearing tokens of at least four characters, sampled without the detector. The detector flags only one of the 60, because most are short numbers, percentages, and version strings such as “25,000” or “2.0.13”. On these, every image arm is poor (0.36–0.45 against 0.96 for text), and allocation neither helps nor hurts (−0.01 [−0.11, 0.08]). Shrinking prose to make room for flagged strings thus did not measurably damage the strings the detector misses, but nothing in the image protects them either.

5.5. Anchors on the allocated image

Anchors [1] copy the flagged strings into the prompt, so their value depends on how many strings compete for the answer. On the single-needle and multi-value tasks of Tab. 1, the only flagged strings in the essay haystack are the needles. The anchor list therefore *is* the answer (1.0 anchor per single-needle document, always the gold value), and K and X reach 0.90–1.00. On multi-key, the list adds three distractor values; recall stays at 0.94–1.00, but readers often return the whole list, and the precision-aware F1 drops to 0.46–0.52. These numbers measure copying rather than reading, so we compare K and X where the list does not reveal the answer.

On the flagged natural identifiers, documents carry 5.3 distinct anchors on average, and choosing one at random would score 0.44. Readers do better (K 0.74, X 0.76) and almost always return a single value, so here the anchors are used, not dumped. Repeating this set at base sizes of 5, 6, and 7 px gives a recall–cost frontier for each arm. X reaches 0.75, 0.76, and 0.81 at 1017, 1128, and 1306 billed tokens, and K reaches 0.71, 0.74, and 0.77 at 1014, 1267, and 1345. X thus lies on or above K’s frontier, by 0.02–0.05 at equal or lower cost (equal at 5 px, where prose cannot shrink). The pooled confidence intervals all include zero, however, and only one reader–budget cell is significant (Tab. 2). At 64K and 128K characters, X costs 31% and 29% fewer billed tokens than K because allocation shrinks the image. Yet on multi-key the precision-aware F1 is the same for S, K, and X (0.36–0.40), so there the anchors add nothing over allocation alone. When the target is *not* flagged, anchors can mislead. On the detector-independent set, X scores 0.36 against 0.44 for S and K (−0.08 vs. K, CI [−0.17, 0.01]), consistent with readers copying a plausible anchor instead of reading the image. On a synthetic sweep of hazard density (Sec. J), anchors on either image become expensive and ineffective

Table 4. **Ordinary comprehension.** LongBench QA F1 on held-out samples (N=50 per task, contexts ≤40K characters, 6 px budget, five readers). */†: paired 95% CI vs. uniform rendering (A) above/below zero. Relative to A, X has no significantly negative cell and S one; every image arm trails plain text T (X−T significantly negative in 9 of 10 cells). Billed tokens: four readers that report usage.

	MultiFieldQA-en					HotpotQA				
Reader	A	S	K	X	T	A	S	K	X	T
Gemini 3.5 Flash-Lite	0.47	0.47	0.53	0.47	0.63	0.59	0.62	0.58	0.57	0.78
GPT-6 Luna	0.36	0.39	0.37	0.36	0.56	0.68	0.73	0.68	0.70	0.72
Qwen3-VL-235B	0.27	0.26	0.31	0.26	0.59	0.66	0.61†	0.65	0.62	0.77
Llama 4 Maverick	0.28	0.25	0.34	0.29	0.59	0.45	0.50	0.51*	0.50	0.68
MiniMax-M3	0.23	0.26	0.25	0.25	0.56	0.29	0.34	0.41*	0.42*	0.69
Mean	0.32	0.33	0.36	0.33	0.58	0.53	0.56	0.57	0.56	0.73
Billed tokens	1141	1056	1440	1355	5342	1200	1114	1239	1153	5617

beyond about 16 flagged strings per 32K characters, which is where routing to text [1] takes over.

5.6. Effect on ordinary comprehension

Shrinking prose could hurt questions that have nothing to do with identifiers. Table 4 tests this on held-out LongBench documents of up to 40K characters. Relative to uniform rendering, F1 is unchanged on MultiFieldQA (0.32 for A, 0.33 for S and X) and slightly higher on HotpotQA (0.53 for A, 0.56 for S and X). X has no significantly negative cell, and S has one (Qwen3-VL on HotpotQA, −0.06). Pooled over readers and tasks, S−A is +0.02 [−0.02, 0.06] and X−A +0.02 [−0.03, 0.07] (Tab. 2), which rules out losses larger than about 0.03 F1. Relative to plain text, every image arm loses. T reaches 0.58 and 0.73, and X trails it by 0.15–0.33 F1 in 9 of 10 cells, at a fifth to a quarter of the text cost. Only GPT-6 Luna reads the HotpotQA images nearly as well as text. Allocation therefore removes a failure specific to identifiers without making optical compression a lossless substitute for text. Whether the remaining comprehension gap is acceptable depends on the reader and the application.

5.7. Analysis

How much does the detector matter? Replacing the regex detector by an oracle that flags exactly the needle values changes nothing when the needles are of a type the detector recognizes (seven-digit numbers: 0.77 vs. 0.78 for S; Tab. 6). When they are not (four-digit numbers, which the detector deliberately ignores), S falls back to uniform rendering in 24 of 25 documents and matches its recall (0.33 vs. 0.32). The oracle would reach 0.82. Together with the detector-independent set of Sec. 5.4, this suggests that missed hazards cost the gain but, in our tests, not more.

Where do real documents fall? Because the routing ratio is computed from text alone, we profiled 32K-character chunks of 13 LongBench tasks, essays, Python source, system logs, and package manifests without querying any model (Sec. L). Prose-dominated documents are routed to images 95–100% of the time, at an estimated 0.19–0.27× the plain-text cost, and technical prose (MultiFieldQA) 75% of the time. Code is mostly routed to text, and logs and manifests always are. Hazard-aware rendering is thus a tool for long documents that *contain* identifiers, not for documents that *consist* of them.

6. Limitations and Conclusion

Limitations. The gains are largest at dense budgets where uniform rendering fails; at 10 px a font search does as well. Optical compression, with or without allocation, trails plain text on ordinary QA by up to 0.33 F1 for weaker readers. The router decides from identifier cost alone, not from this gap. Most fidelity tests use synthetic needles with sparse distractors. Our natural sets are small (60 automatically constructed questions each), the detector protects only high-entropy strings, and anchors can mislead readers when the target is not flagged. We evaluate black-box readers only, through public APIs whose versions and image preprocessing may change, and cannot say how enlarged glyphs change internal attention. Trained optical compressors [8, 41] and token pruning on open models are untested. Budgets follow one patch-based accounting rule (Eq. (1)), but providers tile images differently. Billed savings hold in most settings, with exceptions at short lengths (Gemini up to 32K characters, Qwen at 16K and 5 px, and Llama, whose billing ignores image size). We use ASCII English only, and we do not measure latency, prompt caching [15], or the effect of text inside images bypassing text-side safety filters.

Conclusion. Optical compression decides how many pixels a document receives but not where they go. The renderer reads the text before drawing it, so it can give the strings that must be copied exactly the pixels they need, within the budget of uniform rendering and without access to the model. Across five readers, this raises needle recall by about half the scale at dense budgets, doubles the recall of flagged natural identifiers, and makes the anchors of prior work cheaper where prose can shrink. The gap to plain text on ordinary comprehension remains.

References

[1] Anonymous. Render or retain: Failure-aware optical compression for black-box VLMs. Under review; anonymized copy in the supplementary material, 2026. 2, 3, 4, 5, 6, 7, 8

[2] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. 3, 5

[3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. 3

[4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024. 3, 5

[5] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *Proceedings of the International Conference on Learning Representations*, 2023. 3

[6] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *Proceedings of the European Conference on Computer Vision*, pages 19–35, 2024. 2, 3

[7] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024. 3

[8] Jiale Cheng, Yusen Liu, Xinyu Zhang, Yulin Fei, Wenyi Hong, Ruiliang Lyu, Weihan Wang, Zhe Su, Xiaotao Gu, Xiao Liu, et al. Glyph: Scaling context windows via visual-text compression. *arXiv preprint arXiv:2510.17800*, 2025. 1, 2, 5, 8

[9] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023. 3

[10] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025. 3

[11] Zhengyao Fang, Pengyuan Lyu, Chengquan Zhang, Guangming Lu, Jun Yu, and Wenjie Pei. Prune redundancy, preserve essence: Vision token compression in VLMs via synergistic importance-diversity. In *Proceedings of the International Conference on Learning Representations*, 2026. 3

[12] Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024. 2

[13] Yonghan Gao, Zehong Chen, Lijian Xu, Jingzhi Chen, Jingwei Guan, and Xingyu Zeng. ZeroSense: How vision matters in long context compression. *arXiv preprint arXiv:2603.11846*, 2026. 2

[14] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *Proceedings of the International Conference on Learning Representations*, 2024. 3

[15] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. 3, 8

[16] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Proceedings of the First Conference on Language Modeling*, 2024. 3, 5

[17] Anwen Hu, Haiyang Xu, Liang Zhang, Jiabo Ye, Ming Yan, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mPLUG-DocOwl2: High-resolution compressing for OCR-free multi-page document understanding. *arXiv preprint arXiv:2409.03420*, 2024. 2

[18] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. 3

[19] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. 3

[20] Ivan Yee Lee, Cheng Yang, and Taylor Berg-Kirkpatrick. Optical context compression is just (bad) autoencoding. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[21] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18893–18912, 2023. 2

[22] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025. 3

[23] Jianjian Li, Junquan Fan, Feng Tang, Gang Huang, Shitao Zhu, Songlin Liu, Nian Xie, Wulong Liu, and Yong Liao. FCoT-VL: Advancing text-oriented large vision-language models with efficient visual token compression. *arXiv preprint arXiv:2502.18512*, 2025. 3

[24] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023. 3

[25] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, pages 22947–22970, 2024. 3

[26] Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10564–10578, 2025. 2

[27] Yunhao Liang, Ruixuan Ying, Bo Li, Hong Li, Kai Yan, Qingwen Li, Min Yang, Satoshi Okamoto, Zhe Cui, and Shiwen Ni. Visual merit or linguistic crutch? a close look at DeepSeek-OCR. *arXiv preprint arXiv:2601.03714*, 2026. 2

[28] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024. 3

[29] Yuliang Liu, Biao Yang, Qiang Liu, Zhang Li, Zhiyin Ma, Shuo Zhang, and Xiang Bai. TextMonkey: An OCR-free large multimodal model for understanding document. *arXiv preprint arXiv:2403.04473*, 2024. 2

[30] Guangchuan Lv, Dianxing Shi, and Dingjie Fu. VPRune: Efficient training-free pre-LLM visual token pruning. *arXiv preprint arXiv:2609.24485*, 2026. 3

[31] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, et al. MMLongBench-Doc: Benchmarking long-context document understanding with visualizations. In *Advances in Neural Information Processing Systems*, pages 95963–96010, 2024. 3

[32] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, pages 19327–19352, 2023. 3

[33] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. 3

[34] Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *Proceedings of the International Conference on Learning Representations*, 2023. 1, 2

[35] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22857–22867, 2025. 3

[36] Kaiser Sun, Xiaochuang Yuan, Hongjun Liu, Chen Zhao, Cheng Zhang, Mark Dredze, and Fan Bai. Reading, not thinking: Understanding and bridging the modality gap when text becomes pixels in multimodal LLMs. *arXiv preprint arXiv:2603.09095*, 2026. 2

[37] Michael Tschannen, Basil Mustafa, and Neil Houlsby. CLIPPO: Image-and-language understanding from pixels only. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11006–11017, 2023. 2

[38] Ben Wan, Yan Feng, Zihan Tang, Weizhe Huang, Yuting Zeng, Jia Wang, and Tongxuan Liu. RTPrune: Reading-twice inspired token pruning for efficient DeepSeek-OCR inference. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 124471–124491, 2026. 3

[39] Alex Jinpeng Wang, Linjie Li, Yiqi Lin, Min Li, Lijuan Wang, and Mike Zheng Shou. Leveraging visual tokens for extended text contexts in multi-modal learning. In *Advances in Neural Information Processing Systems*, pages 14325–14348, 2024. 2

[40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3

[41] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025. 1, 2, 5, 8

[42] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026. 2

[43] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *Proceedings of the International Conference on Learning Representations*, 2024. 3

[44] Roy Xie, Dan Friedman, Donghan Yu, Bowen Pan, Christopher Fifty, Jang-Hyun Kim, Xianzhi Du, Zhe Gan, Vivek Rathod, and Bhuwan Dhingra. LensVLM: Selective context expansion for compressed visual representation of text. *arXiv preprint arXiv:2605.07019*, 2026. 2

[45] Ling Xing, Alex Jinpeng Wang, Rui Yan, Xiangbo Shu, and Jinhui Tang. Vision-centric token compression in large language model. In *Advances in Neural Information Processing Systems*, pages 33080–33110, 2025. 2

[46] Jiawei Xu, Zhilin Zhai, Jinrui Fang, Ruohan Xu, Mingfei Lu, Yi Zhang, Guanchu Wang, Tianlong Chen, and Ying Ding. SEER: Long-context reasoning via selective visual-text compression. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[47] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19792–19802, 2025. 2, 3

[48] Shenglai Zeng, Qirui Wang, Kai Guo, Xinnan Dai, Xianxuan Long, and Hui Liu. Magnifying what matters: Attention-guided adaptive rendering for visual text comprehension. *arXiv preprint arXiv:2606.12898*, 2026. 2

[49] Haokui Zhang, Congyang Ou, Dawei Yan, Peng Wang, Qingsen Yan, Yu Zhang, Ying Li, and Rong Xiao. TRIO: Token reduction via inference-objective guidance for efficient vision-language models. *arXiv preprint arXiv:2602.04657*, 2026. 3

[50] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A. Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 74840–74857, 2025. 3

[51] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi

Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, pages 34661–34710, 2023. 3

[52] Hongbo Zhao, Meng Wang, Fei Zhu, Wenzhuo Liu, Bolin Ni, Fanhu Zeng, Gaofeng Meng, and Zhaoxiang Zhang. VTCBench: Can vision-language models understand long context with vision-text compression? *arXiv preprint arXiv:2512.15649*, 2025. 2, 6

[53] FangZhi Zhong, Xuerui Qiu, Yuqi Pan, Ya Liu, Shaowei Gu, Bo Xu, and Guoqi Li. FocusVTC: Efficient and high-performance visual text compression with adaptive resolution. *arXiv preprint arXiv:2609.36651*, 2026. 2

LLM Usage Statement

Large language models are the *objects of study* in this work: the six vision-language models listed in Sec. 4 and the additional Claude reader of Sec. M were queried as black-box readers, and all reported numbers are computed by our scripts from their logged outputs. Every number in the paper is produced programmatically from the experiment logs, and every reference was checked against its primary source. No LLM was used to generate or label evaluation data beyond the readers' answers being evaluated.